



Experiment No: 5

Student Name: Kanishk Bhandari
Branch: BE CSE
Semester: 6th
Subject Name: System Design

UID: 23BCS11100
Section/Group: 23BCS_KRG-2_B
Date of Performance: 18/02/2026
Subject Code: 23CSH-314

1- Aim - Design a messenger application (similar to whatsapp or facebook messenger)

2- Requirements: Functional & Non-Functional

A- Functional Requirement

- Client should be able to register and login himself.
- System should support one-to-one messaging.
- System should also support, group messaging as well.
- System should support sending message in both textual as well as media messages.
- System should support preserving message history.
- Client should be able to see delivered messages (read receipts).

B- Non-Functional Requirement

- Target Scale: 1 Billion users having 100 messages per day which totals into handling 100B messages/day, let's assume: we need 100 TB of storage to store these messages.
- High Availability >>>>> Consistency (Eventual Consistency).
- Latency should be approx 200-300 ms.
- System should be highly reliable. There should not be any packet loss or message drop.

3- Core-entities of System

- User
- Groups
- Messages

4- API endpoint creation

1. Registering a user

1. POST Call: [https://www.whatsapp.com / user / register](https://www.whatsapp.com/user/register)
{
 userName,
 email,
 phoneNumber
}
2. POST Call: [https://www.whatsapp.com / user / login](https://www.whatsapp.com/user/login)

2. One-to-one Message

1. WS: / messages / send
2. GET: / chat / {user_ID}: List <Chat> -> Pagination

3. GET: / messages / {user_ID} / {receiver_ID} [Once the user clicks on a specific chat -> show all messages of that chat]

3.Group-Messages

Create a group

- A. POST: / groups / create

Add Member

- B. POST: / groups / {group_ID} / add

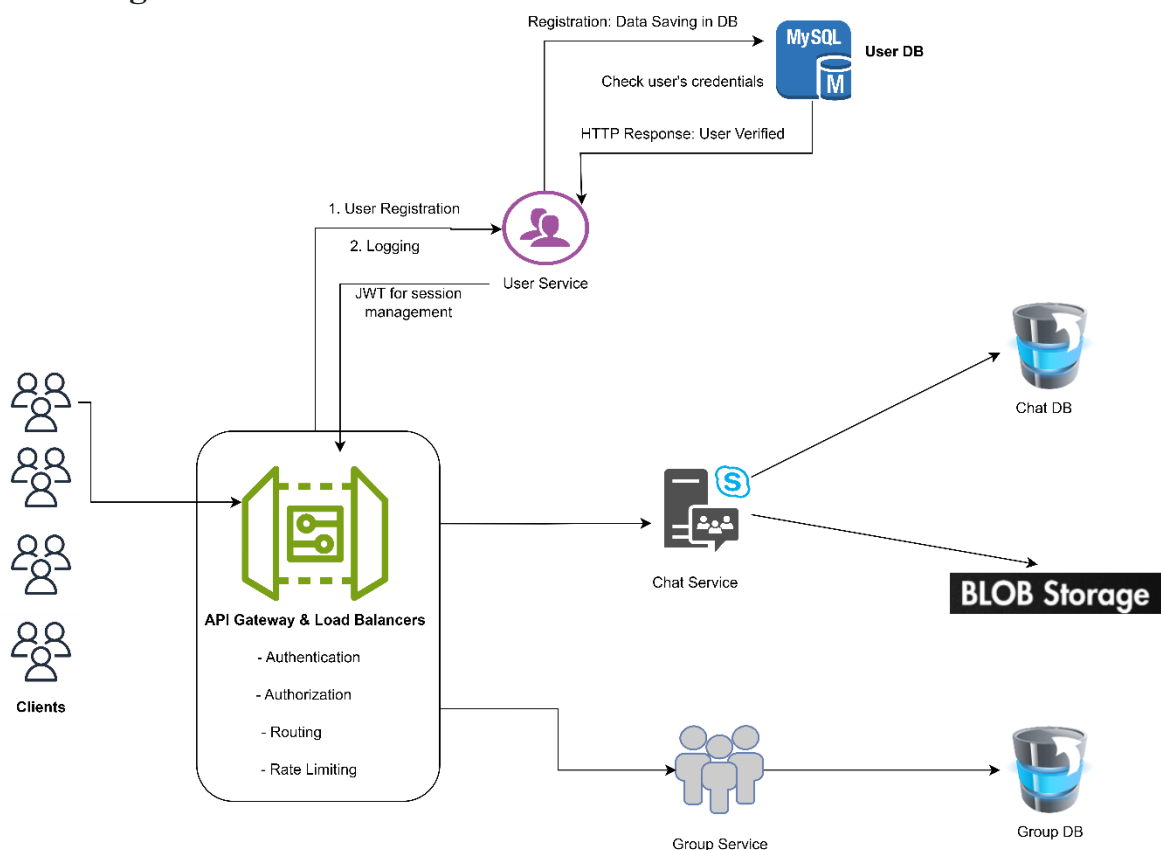
Remove Member

- C. DELETE: / groups / {group_ID} / remove

- A. WS: / messages / send

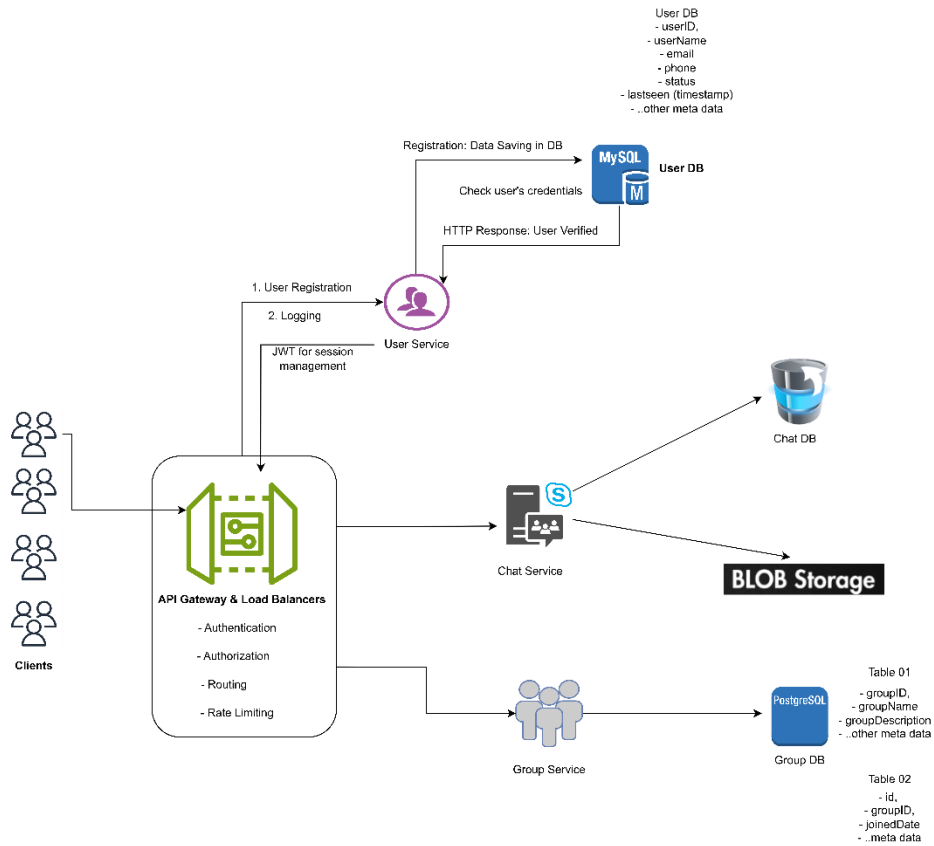
- B. GET: / groups / {group_ID}: List <Chat> -> Pagination

High-Level Design

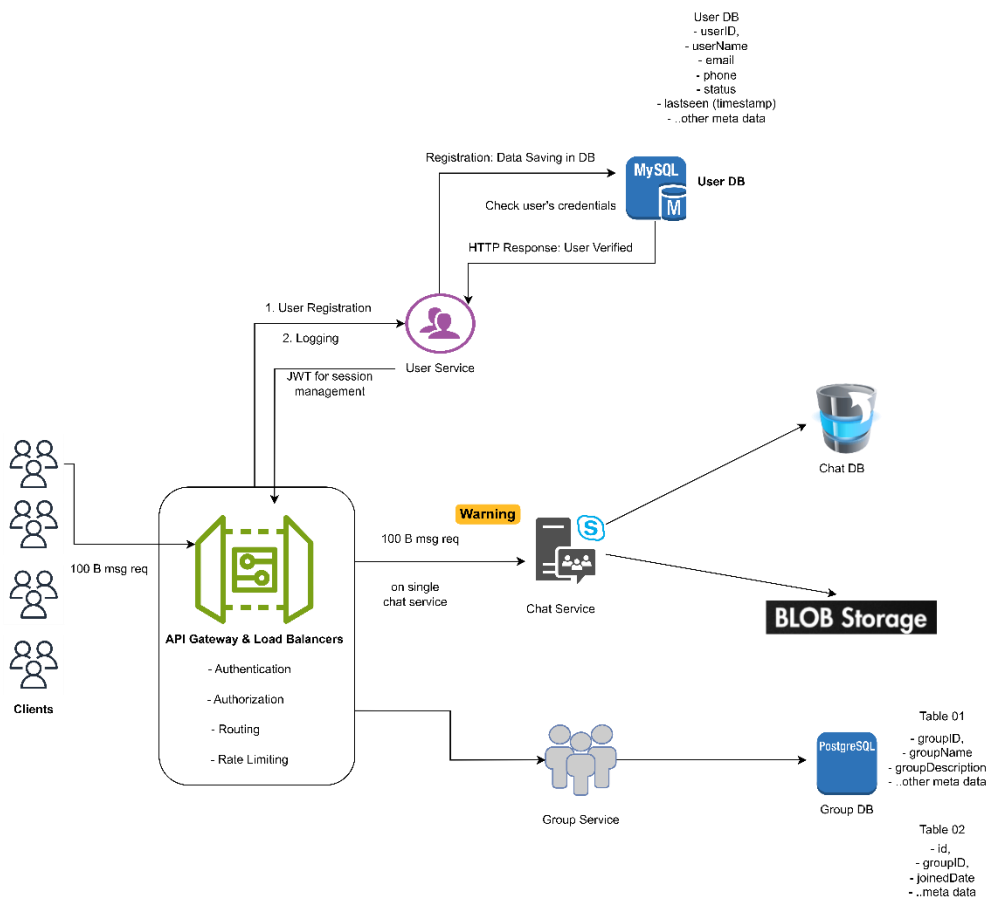


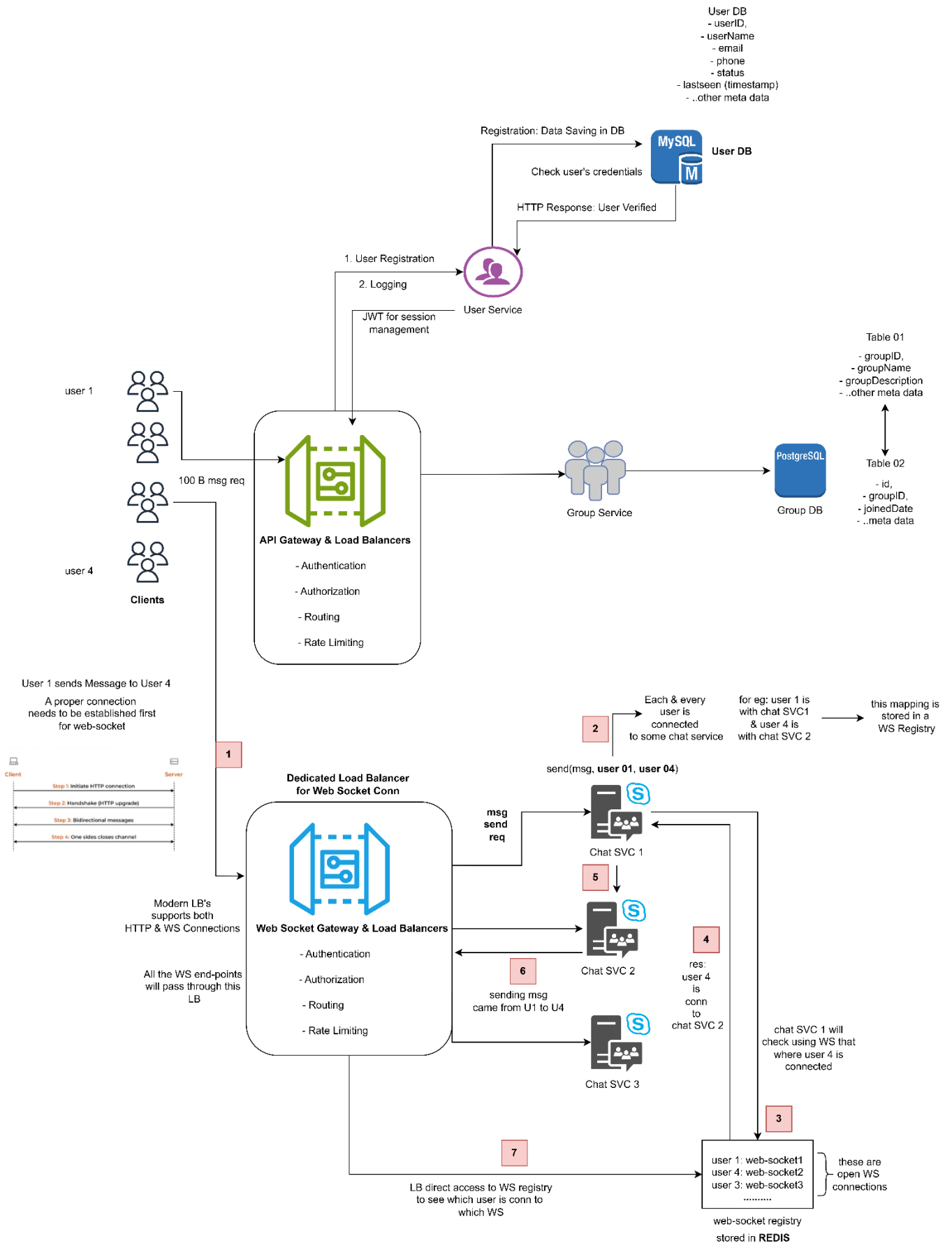
5- Low-Level Design

Group Service



Chat Service





Problem 01: Now, let's suppose, user 01 was connected to WS 1, & in btw due some ntwrk issue the connected got dropped.

in this case, when network come back for user 1, do we create the new WS connection for him???

Solution: Previously made connection will be utilized over here. (Singleton Design Pattern)

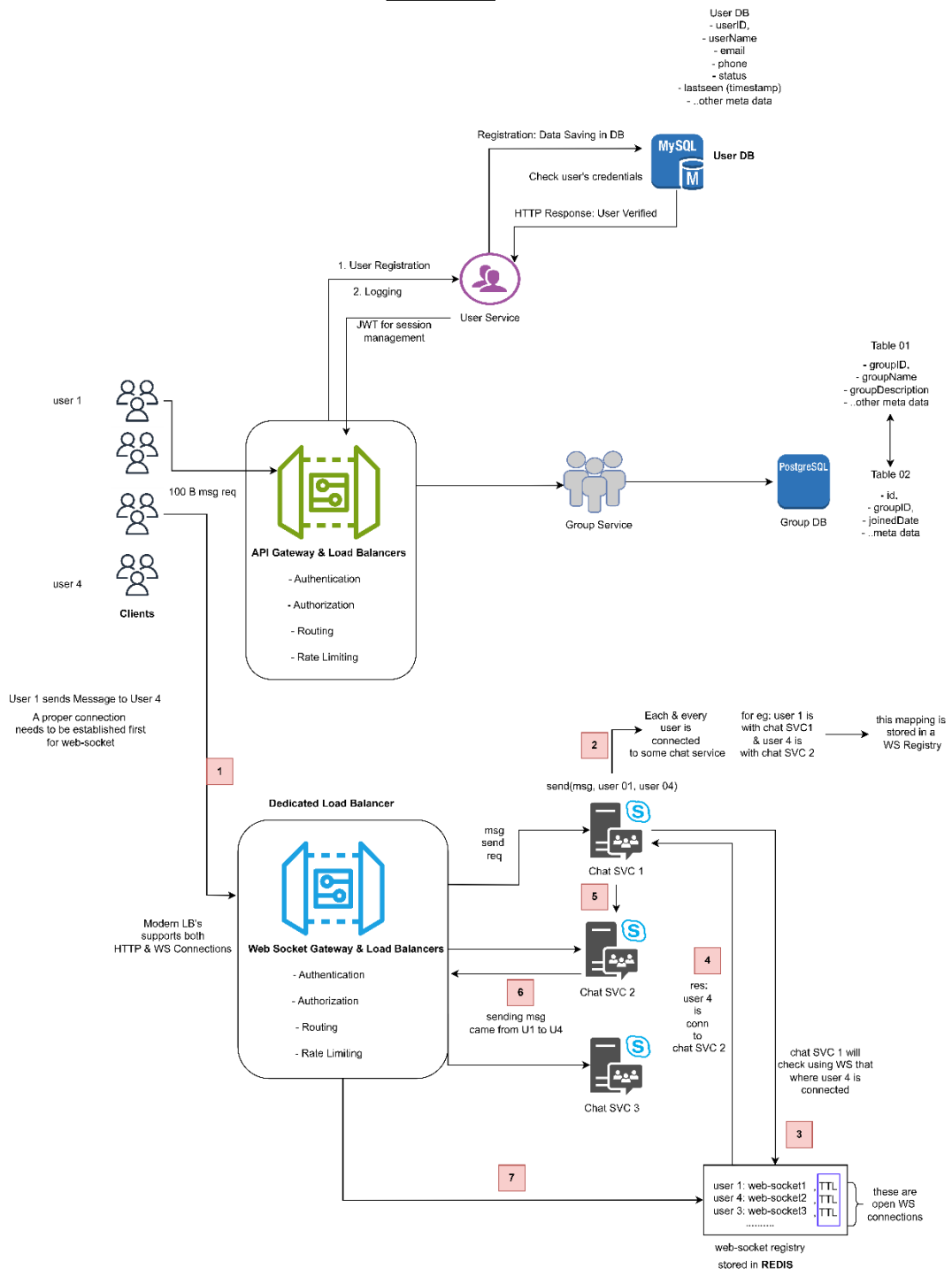
In that case, the LB will have to check intelligently and route the req to previously made connection, thats why it should be directly connected with the WS Registry



Problem 02: Now is it pratically feasible to have the connection open all the time???

on whatsapp, u1 was talking to u2, in btw he left the chat and went for some other task

+
considering the requirement of 1 billion user -> not feasible → we can use TTL for every WS Connection



Problem

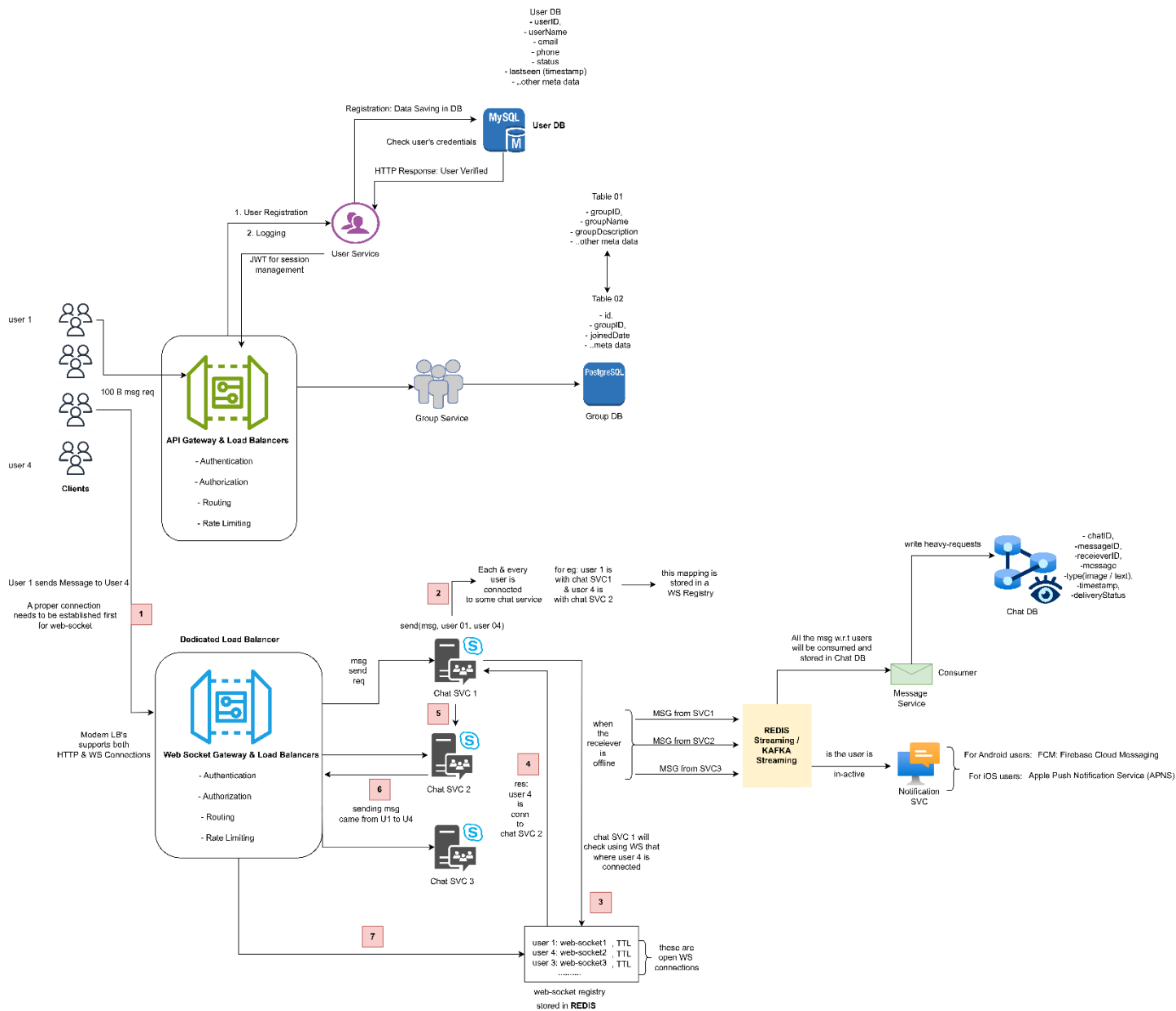
1. Let's say user 1 wants to send the data to user 4, but user 4 is offline OR the connection was dropped in between while they were sharing messages.

2. User 1 was sending a important doc to user 4

A. Since the network was lost: so he will be treated as offline

in that case there should be some mechanism that, when user 4 comes back online he should get the data

and, for that we have to store the data into some database.



Alright so far so good..!
But hang on, do we know that where the images / videos are stored???

whenever a user sending any image / video to other user -> where those images are stored??

For that we have a **uploader service**

